Idempotency Gateway

Pay-Once Protocol

A Complete, Beginner-Friendly Project Explanation & Interview Preparation Guide

Spring Boot · Java 17 · Maven

Prepared for: Apprenticeship Technical Interview & Project Presentation
Author of the project: Nsumba Herve
Document type: Deep teaching walkthrough (7 phases)

How To Use This Document

This document teaches you your own project from zero, the way a patient mentor would teach a junior developer the day before an interview. It does not assume you already understand backend engineering. Every concept is explained **before** it is used, and the reason a thing exists is explained **before** how it works.

The document is organised into seven phases, exactly as your `Explain.md` file requested:

1. **Phase 1 — Project Overview:** What the project does, what idempotency is, and all the core web concepts (REST, HTTP, JSON, status codes, client/server).
2. **Phase 2 — Project Architecture:** The four layers of your application and why they are separated.
3. **Phase 3 — Spring Boot Fundamentals:** Spring Boot, Maven, Tomcat, dependency injection, and every annotation you used.
4. **Phase 4 — Full Code Walkthrough:** Every file, every line, explained slowly.
5. **Phase 5 — Request Flow Simulation:** Three real scenarios traced through the code.
6. **Phase 6 — Mock Interview:** Questions and confident answers.
7. **Phase 7 — 5-Minute Presentation Script:** A word-for-word speaking script.

> **Read this in order.** Each phase builds on the one before it. By Phase 4 you will recognise every word, so the code walkthrough will feel easy instead of overwhelming.

Phase 1 — Project Overview

1.1 What This Project Does (In Real-World Fintech Terms)

Imagine you are using a mobile money app or a card to pay for something — let's say you are buying airtime for 100 Ghana Cedis (GHS). You tap the *"Pay"* button. Behind the scenes, your phone (the **client**) sends a message over the internet to a payment company's computer (the **server**) that says: *"Please charge this customer 100 GHS."*

Now imagine the internet is slow that day. Your phone sends the message, but the reply takes too long to come back. Your phone gives up waiting and thinks: *"The payment failed, let me try again."* So it sends the **same payment message a second time**.

Here is the dangerous part: the first message may have actually **succeeded** on the server — the customer was already charged 100 GHS. The reply was just slow. If the server now processes the second message too, the customer is charged **200 GHS for one purchase**. That is a **double charge**, and in the payment industry it is a serious, money-losing, trust-destroying bug.

Your project is the safety guard that prevents this. It is a small backend service called the *Idempotency Gateway* (you nicknamed it the *"Pay-Once Protocol"*). It sits in front of the payment logic and enforces one simple promise: **no matter how many times the same payment request arrives, the customer is charged exactly once.** Every retry after the first one simply receives a copy of the original answer — without charging again.

> **One-sentence summary you can say in the interview:** "My project is a Spring Boot REST API that makes payment requests safe to retry, so a customer is never accidentally charged twice when the network is unreliable."

1.2 What "Idempotency" Means

"Idempotency" is a slightly scary word for a very simple idea.

An operation is idempotent if doing it many times has the same effect as doing it once.

Everyday examples:

- **A light switch set to "OFF"** — flicking it to OFF when it is already OFF changes nothing. Pressing "OFF" five times leaves the light in exactly the same state as pressing it once. That is idempotent.
- **An elevator "call" button** — pressing it ten times does not make ten elevators come. The result is the same as pressing once. Idempotent.

- **Charging money** — this is *naturally NOT idempotent*. Each charge removes money. Doing it twice removes twice the money. This is exactly the problem.

Because charging money is naturally *not* idempotent, your project's job is to **add idempotency on top of it artificially**. You make a non-idempotent operation (charging) behave like an idempotent one by remembering what you have already done.

> **Interview-ready definition:** "Idempotency means that performing the same operation multiple times produces the same result as performing it once. My gateway makes payments idempotent so retries are safe."

1.3 Why Double Charging Happens

Double charging is almost never caused by a customer clicking twice on purpose. It is caused by the **unreliable nature of the internet**. The three classic causes (also listed in your README) are:

- **Network timeout:** The client sends the request, the server processes it, but the response gets lost or delayed on the way back. The client never hears "success", so it assumes failure and retries.
- **Client retry logic:** Many apps are programmed to automatically resend a request if they don't get a reply quickly. This automatic retry is good for reliability but dangerous for payments.
- **Slow server response:** If the server takes a long time (for example, your project intentionally simulates a 2-second processing delay), the client may lose patience and resend before the first answer arrives.

The crucial insight: **the client cannot tell the difference between "my request was lost" and "my request succeeded but the answer was lost."** From the client's point of view both look identical — silence. So the client retries. The fix cannot live on the client; it must live on the **server**, which *does* know whether it already processed the request.

1.4 Why Payment Systems Use Idempotency Keys

To "remember what was already done", the server needs a way to recognise that two separate requests are really **the same logical payment**. It cannot rely on the request body alone, because two genuinely different customers might both pay "100 GHS" — those are different payments that happen to look identical.

The solution is the **idempotency key**. The client generates a unique identifier (for example a random string like `abc123` or a UUID) *once*, when the user first taps "Pay". The client then attaches that same key to the **first request and to every retry of that same payment**.

The server's logic becomes:

- "Have I seen this key before? **No** → this is a brand-new payment. Process it, then save the key and the result."

- "Have I seen this key before? **Yes** → this is a retry. Do *not* charge again. Just return the saved result."

The key is the thread that ties a request and all of its retries together into one identity. This is exactly how real companies like Stripe, PayPal, and Paystack do it — Stripe famously uses an HTTP header called `Idempotency-Key`, which is the same name your project uses.

1.5 What Problem This System Solves

In one paragraph: **Your system removes the financial risk of network retries.** It lets a client safely retry a payment as many times as it wants, with zero fear of double charging, because the server now has a memory. The first request with a given key does the real work; every later request with that same key gets a fast, free replay of the original answer.

1.6 What Happens During Retry Requests

Walk through it slowly:

1. The client sends payment request #1 with key `abc123`.
2. The server has never seen `abc123`. It processes the payment (in your project: waits 2 seconds to simulate real work), builds the response *"Charged 100.0 GHS"*, and **stores** that response under the key `abc123`.
3. The response is sent back — but suppose it is lost or slow.
4. The client times out and sends retry request #2, with the **same key** `abc123` and the same body.
5. The server looks up `abc123`, finds it already exists, and **does not charge again**. It instantly returns the stored response *"Charged 100.0 GHS"* and adds a special header `X-Cache-Hit: true` to signal "this is a replay, not a fresh charge."

The customer ends up charged once. The client still receives a valid "success" answer. Everyone is happy.

1.7 Why This Is Important In Payment Systems

- **Money correctness:** A double charge is real money lost and a refund process triggered. Idempotency prevents it at the source.
- **Customer trust:** Customers who get double-charged stop using the app. Trust is the whole product in fintech.
- **Regulatory and audit safety:** Payment companies are audited. "We charge exactly once" is a property they must be able to prove.
- **It enables safe retries:** Without idempotency, engineers are afraid to retry. With it, retries become a reliability feature instead of a risk. The network can fail freely and the system still behaves correctly.

1.8 Core Web Concepts You Must Know

Before we look at any code, here is the vocabulary of backend web development. The interviewer may ask any of these directly.

Backend

Software has two halves. The **frontend** is what the user sees and touches — the buttons, screens, colours (a mobile app or a website). The **backend** is the part the user never sees — it runs on a server, handles logic, talks to databases, and makes decisions. **Your project is pure backend.** It has no screens; it only receives requests and sends answers.

Client / Server Architecture

This is the basic shape of almost every internet application.

- The **client** is the program that *asks* for something (a browser, a mobile app, or a testing tool like Postman or `curl`).
- The **server** is the program that *listens* for those asks, does the work, and *answers*.

The client always starts the conversation; the server only ever responds. They talk to each other using a shared language called HTTP.

HTTP

HTTP (HyperText Transfer Protocol) is the agreed-upon set of rules — the "language" — that clients and servers use to talk over the internet. Every HTTP conversation is one **request** from the client followed by one **response** from the server. HTTP is *stateless*: the server does not, by default, remember anything between two requests. (This is exactly why your project must *deliberately* store data — to give the server a memory.)

HTTP Methods & the POST Request

Every HTTP request has a **method** (also called a "verb") that says what kind of action is wanted. The common ones:

	Meaning	Used for
GET	"Give me data"	Reading information (does not change anything)
POST	"Here is data, do something with it"	Creating something / performing an action
PU	"Replace this with	Updating an

T	that"	existing thing
D E L E T E	"Remove this"	Deleting a thing

Your project's main endpoint uses **POST** because a payment is an *action that changes the world* (it moves money). It is not just reading data. POST is the correct verb for "process this payment". POST requests are also the kind that carry a **request body** (explained below).

API Endpoint

An **API** (Application Programming Interface) is the set of "doors" a backend opens so other programs can talk to it. Each individual door is an **endpoint** — a specific URL path combined with an HTTP method. Your project has two endpoints:

- `GET /` — a simple health check that returns the text "Idempotency Gateway running".
- `POST /process-payment` — the real endpoint that processes a payment.

REST API

REST (REpresentational State Transfer) is a popular *style* of designing APIs. A REST API follows a few sensible conventions: it is organised around *resources* (things, named by URLs), it uses the standard HTTP methods (GET/POST/PUT/DELETE) to act on them, it usually exchanges data as JSON, and it uses HTTP status codes to report outcomes. Your project is a small REST API: it exposes resources over HTTP, uses POST, speaks JSON, and returns proper status codes (200 and 409). When the interviewer asks "is this RESTful?" you can say *yes, and explain those four points.*

Request Headers

Every HTTP request has two parts: **headers** and a **body**. Headers are short pieces of *metadata* — "data about the request" — written as name-value pairs. Think of them like the information written on the *outside* of an envelope (sender, postage, handling instructions), while the body is the letter *inside*.

Common headers include `Content-Type: application/json` (telling the server "my body is written in JSON"). Your project uses a custom header, `Idempotency-Key`, to carry the idempotency key. Putting the key in a *header* rather than in the body is intentional: the key is metadata *about* the request, not part of the payment data itself — and it is the same design Stripe uses.

Request Body

The **request body** is the main content the client sends — the actual data the server needs to do its job. For your `POST /process-payment` endpoint, the body carries the payment details: the amount and the currency.

Response Body

The **response body** is the main content the server sends *back*. In your project it is a small JSON object containing a human-readable message, for example `{ "message": "Charged 100.0 GHS" }`.

JSON

JSON (JavaScript Object Notation) is a simple, text-based format for representing structured data. It is the most common language for request and response bodies in modern web APIs because it is easy for both humans and machines to read. It looks like this:

```
{


  "amount": 100,


  "currency": "GHS"


}
```

JSON is built from **key-value pairs** wrapped in curly braces. The keys are text in quotes (`"amount"`), and the values can be numbers (`100`), text (`"GHS"`), true/false, lists, or nested objects. Your backend will automatically convert this JSON text into a Java object, and convert Java objects back into JSON — you will see how in Phase 3 and Phase 4.

HTTP Status Codes

Every HTTP response carries a three-digit **status code** that tells the client, at a glance, what happened. They are grouped by their first digit:

	Meaning	Examples
2xx	Success	**200 OK** — everything worked
3xx	Redirection	301 Moved Permanently

4 x x	Client error (the caller did something wrong)	**409 Conflict**, 400 Bad Request, 404 Not Found
5 x x	Server error (the server itself failed)	500 Internal Server Error

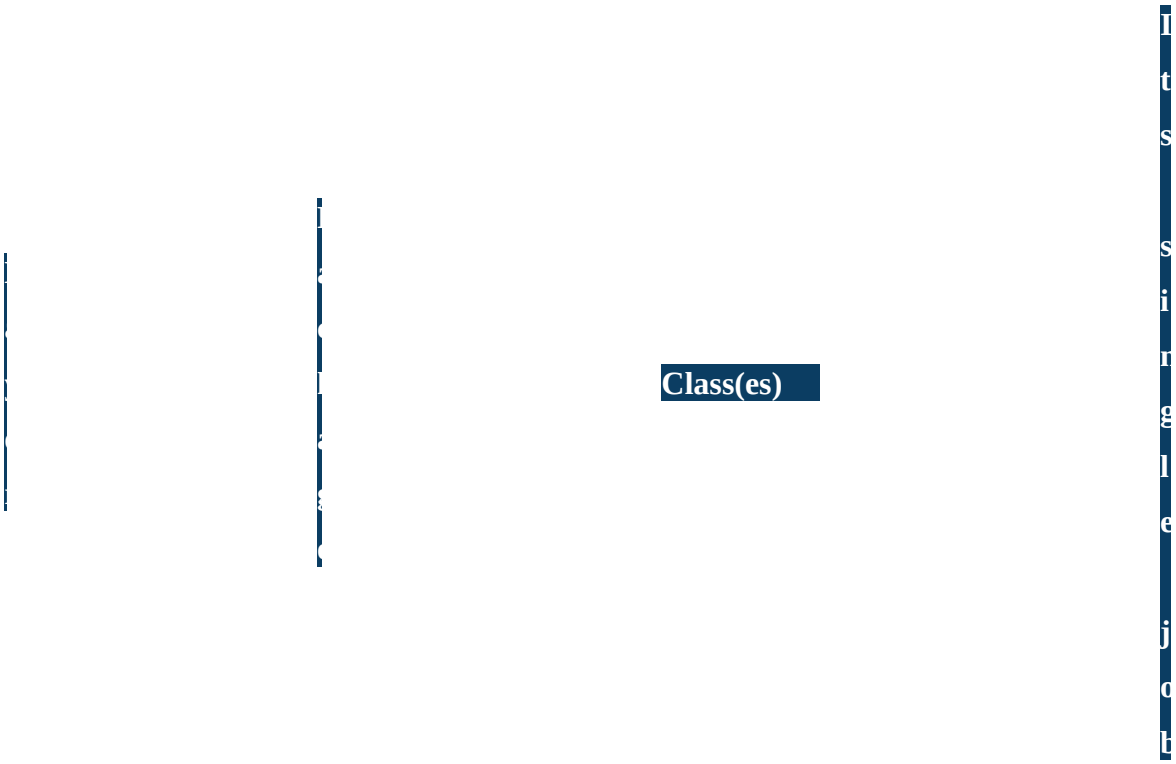
Your project deliberately uses two status codes:

- **200 OK** — for a successful payment, and also for a successful replay of an earlier payment.
- **409 Conflict** — for the dangerous case where someone reuses an idempotency key but sends a *different* payment body. 409 means "your request conflicts with something that already exists" — it is exactly the right code, and choosing it correctly is a great thing to mention in the interview.

Phase 2 — Project Architecture

2.1 The Big Picture

Your project is not one giant file. It is split into **layers**, where each layer has one clear job and hands work to the next. This is called a **layered architecture**, and it is one of the most important professional habits in backend engineering. Your project has four layers plus the application entry point.



E n t r y	i d e m p o	Idempotenc yApplicatio n	S t a r t s

			the whole Spring Boot app
point	tency		
Con	con	PaymentController	Rec

controller

controller

receives

layer

HTTP

requests,

returns

HTTP

re

Service layer	service	PaymentService	Holds the business logic responses

gic (the idempotency rules)

S s PaymentSto S

t o r e

t o r e

re

a v e s

l a y e r

a n d

r e t r i e v e s

d a t a

(t h e

m e m o r y

)

Model layer	model	PaymentRequests, PaymentResponse, StoredPayment, PaymentResult	Defines the shape of the data

t
a

t
h
a
t
m
o
v
e
s

b
e
t
w
e
e
n

l
a
y
e
r
s

2.2 The Controller Layer

The **controller** is the front desk of your application. It is the only layer that knows about the web — about HTTP, headers, status codes, and URLs. Its job is to: (1) receive an incoming HTTP request, (2)

pull out the useful pieces (the idempotency key from the header, the payment data from the body), (3) hand those pieces to the service layer, and (4) take whatever the service returns and wrap it into a proper HTTP response. The controller should contain **no business logic** — it is a translator between "the web world" and "the Java world".

2.3 The Service Layer

The **service** is the brain of your application. This is where the actual *idempotency rules* live. The service does not know or care that it was called over HTTP — it just receives a key and a request object and decides what to do: is this a new payment, a duplicate replay, or a conflict? Keeping logic here (and not in the controller) means the logic could be reused by a different kind of caller — a scheduled job, a test, a command-line tool — without dragging HTTP along with it.

2.4 The Store Layer

The **store** is the memory of your application. HTTP is stateless — the server forgets everything between requests — so to recognise a duplicate the server needs somewhere to remember past payments. The store layer is that memory. In your project the store keeps everything in a `ConcurrentHashMap` held in the computer's RAM. The important architectural idea: the service asks the store to "save this" or "find this" *without knowing how* the store does it. Today it is a map in memory; tomorrow it could be swapped for a real database, and the service would not change a single line. This is the power of separating the store into its own layer.

2.5 The Model Layer

The **model** layer is the vocabulary of your application. It contains plain Java classes that describe the *shape* of your data — what a payment request looks like, what a response looks like, what gets stored. These classes carry data between the other layers. They contain no logic; they are simple data holders. Because their fields match the JSON structure, Spring can automatically convert JSON into these objects and back.

2.6 Why Packages Are Separated

A **package** in Java is just a folder that groups related classes. Your project puts each layer in its own package: `controller`, `service`, `store`, `model`. Why bother?

- **Easy to find things.** Need the web logic? Open `controller`. Need the rules? Open `service`. The folder name tells you the job.
- **Clear boundaries.** The package structure makes the architecture *visible*. A new developer understands the design just by looking at the folders.
- **Controlled dependencies.** Work flows in one direction: controller → service → store. Separated packages make that flow obvious and discourage shortcuts.

2.7 What "Separation of Concerns" Means

A "concern" is one responsibility — one job the software has to do. **Separation of concerns** means each piece of code handles exactly one concern and does not mix in others.

In your project the concerns are cleanly separated: HTTP handling is the controller's concern; the idempotency rules are the service's concern; remembering data is the store's concern; describing data is the model's concern. None of them does another's job.

The opposite — doing everything in one giant class — produces what engineers call a "big ball of mud": hard to read, hard to test, and dangerous to change because one edit can break something unrelated.

2.8 Why Clean Architecture Matters

- **Testability:** You can test the service's idempotency rules without starting a web server, because the rules don't depend on HTTP.
- **Changeability:** Swap the in-memory store for a real database, or add a second endpoint, and the change stays contained in one layer.
- **Readability:** Anyone can open the project and understand it, because each file is small and has one obvious purpose.
- **Teamwork:** Different people can work on different layers at the same time without colliding.

2.9 The Request Flow

Here is the path a request travels through your project:

```
CLIENT (Postman / curl / mobile app) | | HTTP POST /process-payment | Header:
Idempotency-Key: abc123 | Body: { "amount": 100, "currency": "GHS" } v
+-----------------------------------------------+ | PaymentController (Controller
Layer) | | - reads the Idempotency-Key header | | - reads the JSON body into
PaymentRequests | | - calls paymentService.processPayment(...) |
+-----------------------------------------------+ | v
+-----------------------------------------------+ | PaymentService (Service Layer)
| | - asks the store: have I seen this key? | | - NEW key -> wait 2s, build
response, | | save it, return result | | - SAME key+body-> return saved response |
```

```
| (cacheHit = true) | | - SAME key, | | DIFFERENT body-> throw 409 Conflict |
+-----------------------------------------------+               |              v
+-----------------------------------------------+ | PaymentStore (Store Layer) | |
- ConcurrentHashMap in memory | | - exists(key) / get(key) / save(key,payment) |
+-----------------------------------------------+ | v RESPONSE travels back up:
Store -> Service -> Controller | | HTTP 200 OK { "message": "Charged 100.0 GHS" }
| (replays also carry header X-Cache-Hit: true) | (key reused with different body
-> HTTP 409) v CLIENT
```

2.10 Step By Step: What Happens When A Request Enters

1. The client sends an HTTP `POST` to `/process-payment` with the `Idempotency-Key` header and a JSON body.
2. Spring Boot's embedded Tomcat web server receives the raw request and matches the URL and method to the right controller method — `processPayment` in `PaymentController`.
3. Spring extracts the `Idempotency-Key` header value into the `String key` parameter, and converts the JSON body into a `PaymentRequests` Java object.
4. The controller calls `paymentService.processPayment(key, request)`.
5. The service asks the store `exists(key)`. Three branches are possible:
    - **Key not found:** sleep 2 seconds (simulated work), build a `PaymentResponse`, wrap it with the request into a `StoredPayment`, save it under the key, and return a `PaymentResult` with `cacheHit = false`.
    - **Key found, body matches:** return a `PaymentResult` wrapping the saved response with `cacheHit = true`.
    - **Key found, body differs:** throw a `ResponseStatusException` with status `409 Conflict`.
6. The controller receives the `PaymentResult`. If `cacheHit` is true it returns `200 OK` plus the `X-Cache-Hit: true` header; otherwise just `200 OK`. If the service threw the 409 exception, Spring turns it into a `409 Conflict` response automatically.
7. Spring converts the response object back into JSON, and Tomcat sends the HTTP response to the client.

Phase 3 — Spring Boot Fundamentals

3.1 What Is Spring Boot

To understand Spring Boot, first understand **Spring**. Spring is a very large, very popular *framework* for building Java applications. A framework is a collection of pre-written, reusable code that solves the common, boring problems so you can focus on your actual feature. Plain Spring, however, was historically hard to set up — it needed pages of configuration.

Spring Boot is Spring with all the painful setup removed. Its philosophy is "convention over configuration": it makes sensible default choices for you so you can start coding features almost immediately. Spring Boot gives you:

- **Auto-configuration** — it looks at what libraries you included and configures them automatically.
- **An embedded web server** — you don't install a server separately; one is built into your app.
- **Starter dependencies** — bundles of related libraries you add with one line.
- **A simple entry point** — one `main` method runs the whole thing.

Your project uses Spring Boot version 4.0.6 with Java 17.

3.2 What Is Maven

Maven is a *build tool* and *dependency manager* for Java projects. It does two big jobs:

- **Dependency management:** Your project depends on outside libraries (Spring Boot itself, the web library, the test library). You do not download these by hand. You list them in a file called `pom.xml`, and Maven automatically downloads them and all the libraries *they* depend on.
- **Building:** Maven compiles your Java source code, runs your tests, and packages everything into a runnable `.jar` file — all with one command.

The `pom.xml` file (POM = Project Object Model) is the heart of a Maven project. It declares the project's identity (`groupId`, `artifactId`, `version`), its Java version, its parent, and its dependencies. Your project also includes the **Maven Wrapper** (`mvnw` / `mvnw.cmd`) — a small script that lets anyone build the project with the correct Maven version even if they don't have Maven installed. That is why the README says `./mvnw spring-boot:run`.

3.3 What Is Embedded Tomcat

A **web server** is the software that actually listens on a network port, accepts incoming HTTP requests, and sends HTTP responses. **Tomcat** is one of the most widely used Java web servers.

Traditionally you would install Tomcat separately and then "deploy" your app into it. Spring Boot changes this: it **embeds** Tomcat *inside* your application. When you run your project, an entire Tomcat web server starts up automatically inside the same process and begins listening — by default on **port 8080** (which is why your README's base URL is `http://localhost:8080`). You did not write a single line of server code; Spring Boot's auto-configuration set Tomcat up for you because your project includes the web starter dependency.

3.4 What Is Dependency Injection

This is the single most important Spring concept, so we go slowly.

Your classes need each other. `PaymentController` needs a `PaymentService` to do its work. `PaymentService` needs a `PaymentStore`. The question is: *who creates those objects?*

The bad way (without injection): each class builds its own helpers:

```
// inside PaymentController -- the manual way




private final PaymentService paymentService = new
PaymentService(new PaymentStore());
```

This is bad because the controller is now permanently glued to those exact classes. It is hard to test (you cannot substitute a fake service) and the wiring is duplicated everywhere.

The Spring way (dependency injection): a class simply *declares what it needs* in its constructor, and Spring *provides* it:

```
// inside PaymentController -- the Spring way (your actual code)




private final PaymentService paymentService;
```

```
public PaymentController(PaymentService paymentService) {




    this.paymentService = paymentService;   // Spring passes it
in




}
```

Here is how it works. At startup Spring scans your project for classes marked as *components* (with annotations like `@RestController`, `@Service`, `@Component`). For each one it creates a single shared object and keeps it in a big internal registry called the **application context** (sometimes called the "Spring container"). Each managed object is called a **bean**.

When Spring needs to create your `PaymentController`, it sees the constructor asks for a `PaymentService`. Spring looks in its registry, finds the `PaymentService` bean it already built, and passes it in. The same happens for `PaymentService` needing a `PaymentStore`. Spring "injects the dependencies" — hence the name.

A helpful analogy: dependency injection is like a restaurant kitchen. A chef (your controller) does not go farm the vegetables. The chef just says "I need vegetables" and the supply system (Spring) delivers them. The chef focuses on cooking. This is also called **Inversion of Control**: control over object creation is inverted — handed from your code to the framework.

> **Interview gold:** "I used constructor injection. My classes declare their dependencies as constructor parameters and Spring supplies them at startup. This keeps the classes loosely coupled and easy to test."

3.5 What Are Annotations

An **annotation** in Java is a special label, written with an `@` symbol, that you attach to a class, method, or parameter. By itself an annotation does nothing — it is just a tag. Its power comes from the fact that a framework can *read* these tags and change its behaviour because of them.

Think of annotations as sticky notes you put on your code *for Spring to read*. The note `@Service` says to Spring: "treat this class as a service component — create one and manage it." Spring reads the note and acts on it. The annotations are how you communicate your intentions to the framework.

3.6 Every Annotation Used In This Project

@SpringBootApplication

Placed on your main class, `IdempotencyApplication`. It is actually three annotations bundled into one: (1) `@SpringBootConfiguration` — marks this as the configuration root; (2) `@EnableAutoConfiguration` — turns on Spring Boot's automatic setup (this is what makes Tomcat appear, etc.); (3) `@ComponentScan` — tells Spring to scan this package and all sub-packages for component annotations. Because your main class sits in `com.amlitech.idempotency`, Spring scans `controller`, `service`, `store`, and `model` automatically.

@RestController

Placed on `PaymentController`. It tells Spring two things. First, "this class handles incoming web requests" — so register its methods as endpoints. Second, "whatever these methods return should be sent *directly* as the response body" — converted to JSON if it is an object. It is the combination of two older annotations, `@Controller` + `@ResponseBody`.

@Service

Placed on `PaymentService`. Technically it does the same job as `@Component` — it marks the class as a Spring-managed bean. But it carries *meaning*: it documents to any reader "this class holds business logic; it belongs to the service layer." Spring will create one `PaymentService` bean and inject it wherever a `PaymentService` is needed.

@Component

Placed on `PaymentStore`. `@Component` is the generic, most basic "this is a Spring-managed bean" marker. `@Service`, `@RestController`, and `@Repository` are all specialised versions of it. The store is marked with the plain `@Component`; Spring creates one shared `PaymentStore` and injects it into `PaymentService`.

@GetMapping

Placed on the `home()` method. It maps HTTP `GET` requests for a given URL path to that method. `@GetMapping("/")` means "when a GET request arrives for the path `/`, run this method."

@PostMapping

Placed on the `processPayment()` method. It maps HTTP `POST` requests for a path to that method. `@PostMapping("/process-payment")` means "when a POST request arrives for `/process-payment`, run this method." POST is correct here because processing a payment is an action that changes state.

@RequestHeader

Placed on a method parameter. It tells Spring "fill this parameter with the value of a named HTTP

header." `@RequestHeader("Idempotency-Key") String key` means: read the `Idempotency-Key` header from the incoming request and put its text into the `key` variable. If the header is missing, Spring rejects the request automatically.

@RequestBody

Placed on a method parameter. It tells Spring "take the JSON in the request body and convert it into this Java object." `@RequestBody PaymentRequests request` means: read the JSON body, and using a library called Jackson, build a `PaymentRequests` object whose fields are filled from the JSON. This automatic JSON-to-object conversion is called **deserialization**.

ResponseEntity (a class, not an annotation)

`ResponseEntity` is a Spring class that represents a *complete* HTTP response: the status code, the headers, *and* the body, all together. You use it when you want precise control over the response — which your project does, because for a cache hit you must add the custom `X-Cache-Hit` header. `ResponseEntity.ok(body)` builds a 200 OK response with that body; `ResponseEntity.ok().header("X-Cache-Hit","true").body(...)` builds a 200 OK response that also carries the extra header.

3.7 How Spring Boot Starts (The Lifecycle)

1. You run the app. The JVM calls the `main` method in `IdempotencyApplication`.
2. `SpringApplication.run(...)` bootstraps Spring — it creates the application context (the bean container).
3. Component scanning runs. Spring finds `PaymentController`, `PaymentService`, `PaymentStore` by their annotations.
4. Spring creates the beans in dependency order: `PaymentStore` first, then `PaymentService` (injecting the store), then `PaymentController` (injecting the service).
5. Auto-configuration starts the embedded Tomcat server on port 8080.
6. The app is now live and waiting for HTTP requests.

Phase 4 — Full Code Walkthrough

Now we examine every file. For each one: what it is, why it exists, where it sits, who calls it, and every line explained. We move in the order a request flows.

4.1 pom.xml — The Project Definition

What it is: the Maven configuration file. It is not Java code; it is XML that describes the project to the build tool. **Why it exists:** it tells Maven the project's identity, which Java version to use, and which libraries to download. **Where it fits:** it is the foundation — before any Java runs, Maven reads this to assemble the project.

```
<parent>



    <groupId>org.springframework.boot</groupId>



    <artifactId>spring-boot-starter-parent</artifactId>



    <version>4.0.6</version>



</parent>
```

The **parent** says "inherit settings from Spring Boot's standard parent project, version 4.0.6." This is what gives you sensible defaults and lets you list dependencies *without writing their version numbers* — the parent already knows the correct, compatible versions.

```
<groupId>com.amlitech</groupId>



<artifactId>idempotency</artifactId>



<version>0.0.1-SNAPSHOT</version>
```

These three lines are the project's unique identity. `groupId` is the organisation, `artifactId` is the project name, `version` is the version (`SNAPSHOT` means "still in development").

```
<properties>



    <java.version>17</java.version>



</properties>
```

Sets the Java version to **17** — a modern, long-term-support release.

```
<dependency>



    <groupId>org.springframework.boot</groupId>



    <artifactId>spring-boot-starter-webmvc</artifactId>
```

```
</dependency>
```

This is the most important dependency. It is the **web starter** — a bundle that brings in everything needed to build a REST API: Spring MVC (the request-routing engine), the embedded Tomcat server, and Jackson (the JSON converter). One line, and your project can speak HTTP. (Note: in Spring Boot 4 this starter is named `spring-boot-starter-webmvc`; in older versions it was `spring-boot-starter-web` — mention this if asked.)

```
<dependency>



    <groupId>org.springframework.boot</groupId>



    <artifactId>spring-boot-devtools</artifactId>



    <scope>runtime</scope>



    <optional>true</optional>



</dependency>
```

DevTools is a developer-convenience library — it automatically restarts the app when you change code, so you don't restart manually while developing.

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>



    <artifactId>spring-boot-starter-webmvc-test</artifactId>



    <scope>test</scope>



</dependency>
```

The **test starter** — brings in testing libraries (JUnit and Spring's test support). `scope=test` means it is only used when running tests, never shipped in the final product.

```
<build>



    <plugins>



        <plugin>



            <groupId>org.springframework.boot</groupId>



            <artifactId>spring-boot-maven-plugin</artifactId>
```

```
        </plugin>



    </plugins>



</build>
```

The **Spring Boot Maven plugin** — it packages your app into a single runnable JAR and powers the `./mvnw spring-boot:run` command.

4.2 IdempotencyApplication.java — The Entry Point

What it is: the class that starts your whole application. **Why it exists:** every Java program needs a `main` method as its starting point; this is yours. **Where it fits:** it sits at the top of the package tree so component scanning can reach every other class. **Who calls it:** the Java Virtual Machine calls `main`; nothing in your code calls it.

```
package com.amlitech.idempotency;









import org.springframework.boot.SpringApplication;



import
org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication



public class IdempotencyApplication {









    public static void main(String[] args) {



        SpringApplication.run(IdempotencyApplication.class,
args);



    }



}
```

Line by line:

- `package com.amlitech.idempotency;` — declares which package (folder) this class lives in. This is the *root* package; everything else is below it.
- The two `import` lines bring in the Spring classes this file uses: `SpringApplication` (the launcher) and `SpringBootApplication` (the annotation).
- `@SpringBootApplication` — the master annotation explained in Phase 3: it enables auto-configuration and component scanning.
- `public class IdempotencyApplication` — declares the class. `public` means visible everywhere.
- `public static void main(String[] args)` — the standard Java entry point. `static` means it can run without creating an object first; `void` means it returns nothing; `args` holds any command-line arguments.
- `SpringApplication.run(IdempotencyApplication.class, args);` — the one line that

does everything: it starts Spring, builds the application context, creates all beans, and launches embedded Tomcat. After this line, your app is alive.

4.3 The Model Layer — The Data Shapes

We explain the models before the controller and service, because the controller and service constantly use them — you need this vocabulary first. All four are **POJOs** (Plain Old Java Objects): simple classes with private fields, a no-argument constructor, and getters/setters. They contain data, not logic.

Why getters, setters, and a no-arg constructor?

Two reasons. First, **encapsulation** — a core object-oriented principle: fields are kept `private` so outside code cannot touch them directly; access goes through `getX()` (read) and `setX()` (write) methods. This gives the class control over its own data. Second, **Jackson** — the JSON library Spring uses — needs a no-argument constructor to create an empty object, and then calls the setters to fill in each field from the JSON. So this pattern is what makes automatic JSON conversion possible.

4.3.1 PaymentRequests.java — the incoming data

What it is: a model that represents the payment data the client sends. **Why it exists:** the JSON body `{ "amount": 100, "currency": "GHS" }` needs a matching Java shape to be converted into. **Who uses it:** Spring builds one from the request body via `@RequestBody`; the service reads it.

```
package com.amlitech.idempotency.model;









public class PaymentRequests {



    private double amount;



    private String currency;
```

```
    public PaymentRequests(){



    }









    public double getAmount() { return amount; }



    public void setAmount(double amount) { this.amount = amount;
}









    public String getCurrency() { return currency; }



    public void setCurrency(String currency) { this.currency =
currency; }



}
```

- `private double amount;` — the payment amount. `double` is a number type that allows

decimals (e.g. 100.0, 49.99).

- `private String currency;` — the currency code, e.g. "GHS", as text.
- The empty constructor `PaymentRequests(){}` — required by Jackson to create the object before filling it.
- The getters and setters — Jackson sees the JSON key `"amount"` and calls `setAmount(100)`; it sees `"currency"` and calls `setCurrency("GHS")`. The field names *match* the JSON keys, which is how the mapping works.

> **Honest detail to mention if asked:** the class name is plural ("PaymentRequests") although it represents a single request — "PaymentRequest" would read more naturally. It is purely a naming choice and does not affect behaviour.

4.3.2 PaymentResponse.java — the outgoing data

What it is: the model that represents the answer sent back to the client. **Why it exists:** the client expects a JSON reply like `{ "message": "Charged 100.0 GHS" }`; this class is its Java shape. **Who uses it:** the service creates it; Spring converts it back into JSON for the response body.

```
package com.amlitech.idempotency.model;









public class PaymentResponse {



    private String message;
```

```
    public PaymentResponse() {}









    public PaymentResponse(String message) {



        this.message = message;



    }









    public String getMessage() { return message; }



    public void setMessage(String message) { this.message =
message; }



}
```

- `private String message;` — the single field, a human-readable result text.
- It has **two** constructors. The empty one is for Jackson. The second, `PaymentResponse(String message)`, is a convenience the service uses to build a response in one line: `new PaymentResponse("Charged 100.0 GHS")`.
- When this object is returned, Jackson reads `getMessage()` and produces the JSON

`{ "message": "..." }`. Turning an object into JSON is called **serialization** — the opposite of deserialization.

4.3.3 StoredPayment.java — the saved record

What it is: a model that bundles together *both* the original request and the response that was produced. **Why it exists:** this is the heart of idempotency. To handle a duplicate correctly the store must remember two things: (1) the response, so it can replay it; and (2) the original request body, so it can check a retry truly matches and is not a different payment reusing the key. `StoredPayment` holds both. **Who uses it:** the service creates it and saves it; the store keeps it; the service reads it back on a duplicate.

```
package com.amlitech.idempotency.model;









public class StoredPayment {



    private PaymentRequests request;



    private PaymentResponse response;









    public StoredPayment() {}
```

```
    public StoredPayment(PaymentResponse response,
PaymentRequests request) {

        this.response = response;

        this.request = request;

    }

    public PaymentRequests getRequest() { return request; }

    public void setRequest(PaymentRequests request)
{ this.request = request; }

    public PaymentResponse getResponse() { return response; }
```

```
    public void setResponse(PaymentResponse response)
{ this.response = response; }




}
```

- `private PaymentRequests request;` — the original request, kept so future retries can be compared against it.
- `private PaymentResponse response;` — the response that was generated, kept so it can be replayed instantly.
- The constructor takes both and stores them. Notice the parameter order is `(response, request)` — be careful and consistent when calling it.

4.3.4 PaymentResult.java — the internal carrier

What it is: a small model used *only inside the application* to carry two things from the service back to the controller: the response, and a true/false flag saying whether this was a cache hit (a replay). **Why it exists:** the controller needs to know whether to add the `X-Cache-Hit` header. The `PaymentResponse` alone cannot tell it that. `PaymentResult` wraps the response together with the `cacheHit` flag. **Who uses it:** the service returns it; the controller reads it. It is never converted to JSON itself — only its inner `response` is.

```
package com.amlitech.idempotency.model;









public class PaymentResult {



    private PaymentResponse response;
```

```
    private boolean cacheHit;









    public PaymentResult() {}









    public PaymentResult(PaymentResponse response, boolean
cacheHit) {



        this.response = response;



        this.cacheHit = cacheHit;



    }









    public PaymentResponse getResponse() { return response; }
```

```
    public void setResponse(PaymentResponse response)
{ this.response = response; }









    public boolean isCacheHit() { return cacheHit; }



    public void setCacheHit(boolean cacheHit) { this.cacheHit =
cacheHit; }



}
```

- `private PaymentResponse response;` — the actual answer for the client.
- `private boolean cacheHit;` — a `boolean` (true/false) flag. `true` means "this answer is a replay of a stored payment"; `false` means "this is a freshly processed payment."
- Notice the getter for the boolean is `isCacheHit()`, not `getCacheHit()`. By Java convention, boolean getters start with `is`.

> **Design point worth saying aloud:** "I separated `PaymentResult` from `PaymentResponse` so that the cache-hit flag — which is internal control information — never leaks into the JSON the client sees. The client gets a clean response; the controller gets the extra flag it needs."

4.4 PaymentStore.java — The Memory Layer

What it is: the class that stores and retrieves payment records. **Why it exists:** HTTP is stateless — without a store the server cannot recognise a duplicate key. This class *is* the server's memory. **Where it fits:** the lowest layer; nothing is below it. **Who calls it:** only `PaymentService`. **Who it calls:** nothing — it just manages a map.

```
package com.amlitech.idempotency.store;

import com.amlitech.idempotency.model.PaymentResponse;

import com.amlitech.idempotency.model.StoredPayment;

import org.springframework.stereotype.Component;

import java.util.concurrent.ConcurrentHashMap;

@Component

public class PaymentStore {
```

```
    private final ConcurrentHashMap<String, StoredPayment> store
= new ConcurrentHashMap<>();









    public StoredPayment get(String key){



        return store.get(key);



    }



    public void save(String key, StoredPayment payment){



        store.put(key, payment);



    }



    public boolean exists(String key){



        return store.containsKey(key);
```

```
    }



}
```

The imports: `StoredPayment` is the value type the map holds; `Component` is the Spring annotation; `ConcurrentHashMap` is the data structure. (`PaymentResponse` is imported but not actually used — a harmless leftover.)

`@Component` — marks this as a Spring bean. Spring creates one shared `PaymentStore` at startup and injects it into `PaymentService`. Because there is exactly one shared instance, there is exactly one shared memory — which is what you want.

The field:

```
private final ConcurrentHashMap<String, StoredPayment> store =
new ConcurrentHashMap<>();
```

- A **Map** is a dictionary-like structure of *key → value* pairs. Here the key is a `String` (the idempotency key) and the value is a `StoredPayment` (the saved record).
- `final` means the `store` variable will always point to this same map — you can change what is *inside* the map, but you cannot replace the map itself. This prevents accidental bugs.
- It is created immediately with `new ConcurrentHashMap<>()`, so the store is ready the moment the bean exists.

Why ConcurrentHashMap and not a plain HashMap?

This is a key interview point. A web server is **multi-threaded** — it handles many requests at the same time, each on its own thread. Two requests could try to read and write the map at the very same instant.

A plain `HashMap` is **not thread-safe**: simultaneous access can corrupt its internal structure, causing wrong results or even crashes. A `ConcurrentHashMap` is **thread-safe** — it is specifically designed for many threads to use it at once, safely and efficiently. Since your gateway is exactly the kind of system that receives bursts of concurrent (often duplicate) requests, `ConcurrentHashMap` is the correct, deliberate choice.

The three methods

- `get(String key)` — **input:** a key. **output:** the `StoredPayment` saved under it, or `null` if nothing is there. It simply delegates to `store.get(key)`.
- `save(String key, StoredPayment payment)` — **input:** a key and a record. **output:** none

(`void`). It calls `store.put(key, payment)` to insert or overwrite the entry.

- `exists(String key)` — **input:** a key. **output:** a `boolean` — `true` if the map already contains that key, `false` otherwise. It calls `store.containsKey(key)`. This is the method the service uses to decide "new payment or duplicate?"

Why in-memory storage instead of a database?

The store keeps data in RAM, in the map. A real production system would use a fast database such as **Redis** (with an expiry time on each key). For this project, in-memory was chosen because: it keeps the project simple and focused on demonstrating the *idempotency logic itself*, not database setup; it is extremely fast; and it requires no external software to run or grade. The honest trade-off, which you should state proudly because it shows awareness: **in-memory data is lost when the app restarts**, and it is not shared if you run several copies of the app. Because the store is its own isolated layer, upgrading to Redis later would change only this one file — the service and controller would not change at all.

4.5 PaymentService.java — The Brain

What it is: the class holding the idempotency business logic. **Why it exists:** to keep the decision-making rules in one focused place, separate from web concerns. **Where it fits:** the middle layer. **Who calls it:** `PaymentController`. **Who it calls:** `PaymentStore`.

```
package com.amlitech.idempotency.service;









import com.amlitech.idempotency.model.PaymentRequests;



import com.amlitech.idempotency.model.PaymentResponse;



import com.amlitech.idempotency.model.PaymentResult;
```

```
import com.amlitech.idempotency.model.StoredPayment;



import com.amlitech.idempotency.store.PaymentStore;



import org.springframework.http.HttpStatus;



import org.springframework.stereotype.Service;



import org.springframework.web.server.ResponseStatusException;









@Service



public class PaymentService {









    private final PaymentStore paymentStore;
```

```
    public PaymentService(PaymentStore paymentStore) {

        this.paymentStore = paymentStore;

    }
```

The imports bring in the four model classes, the `PaymentStore`, and three Spring items: `HttpStatus` (an enum of HTTP status codes), `Service` (the annotation), and `ResponseStatusException` (an exception that carries an HTTP status).

`@Service` — marks this as a Spring-managed service bean.

The field and constructor — this is constructor-based dependency injection. The service declares `private final PaymentStore paymentStore;` and receives it through the constructor. Spring sees the constructor needs a `PaymentStore`, finds the one bean it created, and passes it in. `final` guarantees the reference never changes after construction.

The processPayment method — the core of the whole project

```
public PaymentResult processPayment(String key, PaymentRequests
requests){

    if(paymentStore.exists(key)){

        StoredPayment saved = paymentStore.get(key);
```

```
        if (saved.getRequest().getAmount() ==
requests.getAmount()




                &&
saved.getRequest().getCurrency().equals(requests.getCurrency()))
{




            return new PaymentResult(saved.getResponse(), true);




        }else{




            throw new ResponseStatusException(




                HttpStatus.CONFLICT,




                "Idempotency key already used for a different
request body."




            );




        }




    }
```

```
    try{



        Thread.sleep(2000);



    }catch (InterruptedException e){



        e.printStackTrace();



    }



    PaymentResponse response = new PaymentResponse("Charged " +
requests.getAmount() + " " + requests.getCurrency());



    StoredPayment payment = new StoredPayment(response,
requests);



    paymentStore.save(key, payment);









    return new PaymentResult(response, false);
```

```
}
```

Inputs: `key` (the idempotency key from the header) and `requests` (the payment data from the body). **Output:** a `PaymentResult` — *or* a thrown `409` exception.

Now line by line:

- `if(paymentStore.exists(key))` — the very first decision: **have I seen this key before?** If yes, we are in the "duplicate" branch. If no, we skip to the "new payment" branch lower down.
- **Inside the duplicate branch:** `StoredPayment saved = paymentStore.get(key);` — fetch the record we saved last time, so we can compare and replay it.
- The inner `if` is the **conflict check**. It compares the saved request with the new one on two fields:
  - `saved.getRequest().getAmount() == requests.getAmount()` — do the amounts match? `==` compares the two `double` numbers.
  - `saved.getRequest().getCurrency().equals(requests.getCurrency())` — do the currencies match? Note: `String` values must be compared with `.equals()`, never `==`, because `==` would compare object identity rather than text content. Using `.equals()` here is correct.
  - The `&&` means **both** conditions must be true for the bodies to be considered "the same".
- **If both match** — this is a genuine retry of the same payment. `return new PaymentResult(saved.getResponse(), true);` — return the *stored* response with `cacheHit = true`. Crucially: **no charge happens, no 2-second sleep happens.** The reply is instant. This is the "cache hit" / "replay" behaviour.
- **If they do not match** — someone reused a key that already belongs to a different payment. That is a dangerous mistake, so we refuse: `throw new ResponseStatusException(HttpStatus.CONFLICT, "...")`. Throwing this exception immediately stops the method. Spring catches it and turns it into an HTTP **409 Conflict** response with that message. This protects against a client bug or a misuse where one key would otherwise mask a totally different transaction.
- **The new-payment branch** (only reached when the key was *not* found):
  - `Thread.sleep(2000);` — pauses this thread for 2000 milliseconds (2 seconds). This is a **deliberate simulation** of slow, real payment work (contacting a bank, etc.). It also makes the demo realistic: it creates the very window of time in which a client would time out and retry — so you can *show* the idempotency working.
  - It is wrapped in `try/catch` because `Thread.sleep` can throw a checked `InterruptedException`, which Java forces you to handle. The `catch` simply prints the error trace.
  - `PaymentResponse response = new PaymentResponse("Charged " + requests.getAmount() + " " + requests.getCurrency());` — builds the success message by joining text with the amount and currency, e.g. `"Charged 100.0 GHS"`.
  - `StoredPayment payment = new StoredPayment(response, requests);` — bundles the response together with the original request into one record.

- `paymentStore.save(key, payment);` — **saves the record under the key.** This is the single most important line for idempotency: it is what gives the server its memory. From this moment on, any retry with this key will be recognised as a duplicate.
- `return new PaymentResult(response, false);` — returns the fresh response with `cacheHit = false` (this was a real, first-time charge).

> **The whole method in one sentence:** "If the key is new, do the real work, save it, and report a fresh charge; if the key is known and the body matches, replay the saved answer for free; if the key is known but the body differs, reject with 409."

4.6 PaymentController.java — The Front Desk

What it is: the class that exposes the HTTP endpoints. **Why it exists:** to be the bridge between the web and the service layer. **Where it fits:** the top layer, closest to the client. **Who calls it:** Spring (which is triggered by an incoming HTTP request). **Who it calls:** `PaymentService`.

```
package com.amlitech.idempotency.controller;









import com.amlitech.idempotency.model.PaymentRequests;



import com.amlitech.idempotency.model.PaymentResponse;



import com.amlitech.idempotency.model.PaymentResult;



import com.amlitech.idempotency.service.PaymentService;
```

```
import org.springframework.http.ResponseEntity;



import org.springframework.web.bind.annotation.*;









@RestController



public class PaymentController {









    private final PaymentService paymentService;









    public PaymentController(PaymentService paymentService) {



        this.paymentService = paymentService;
```

```
    }









    @GetMapping("/")



    public String home(){



        return "Idempotency Gateway running";



    }









    @PostMapping("/process-payment")



    public ResponseEntity<PaymentResponse> processPayment(



            @RequestHeader("Idempotency-Key") String key,
```

```
            @RequestBody PaymentRequests request) {









        PaymentResult result =
paymentService.processPayment(key, request);









        if (result.isCacheHit()) {



            return ResponseEntity.ok()



                    .header("X-Cache-Hit", "true")



                    .body(result.getResponse());



        }
```

```
        return ResponseEntity.ok(result.getResponse());




    }



}
```

The imports bring in the models it uses, the `PaymentService`, `ResponseEntity`, and (with the `*` wildcard) all the web annotations.

`@RestController` — marks this as a REST controller: its methods become HTTP endpoints, and their return values become the response body.

Field and constructor — constructor injection again: the controller declares it needs a `PaymentService`, and Spring injects the service bean.

The `home()` method:

- `@GetMapping("/")` — handles `GET /`.
- It returns the plain string `"Idempotency Gateway running"`. This is a simple **health check** — you can open `http://localhost:8080/` in a browser and instantly confirm the app is up.

The `processPayment()` method — the real endpoint:

- `@PostMapping("/process-payment")` — handles `POST /process-payment`.
- Return type `ResponseEntity<PaymentResponse>` — it returns a full HTTP response whose body is a `PaymentResponse`. `ResponseEntity` is used (instead of returning the object directly) because the controller needs to control the headers.
- `@RequestHeader("Idempotency-Key") String key` — Spring reads the `Idempotency-Key` header and gives its text to `key`. If the header is missing, Spring returns a `400 Bad Request` automatically — the key is mandatory.
- `@RequestBody PaymentRequests request` — Spring converts the JSON body into a `PaymentRequests` object via Jackson (deserialization).
- `PaymentResult result = paymentService.processPayment(key, request);` — the controller delegates **all the thinking** to the service. Notice the controller itself contains no idempotency logic — that is separation of concerns in action. If the service throws the 409 exception, the controller never reaches the next line; Spring handles it.
- `if (result.isCacheHit())` — checks the flag the service set. If it is a replay:
    - `ResponseEntity.ok()` — start building a 200 OK response.
    - `.header("X-Cache-Hit", "true")` — attach the custom header signalling "this is a replayed response, not a fresh charge."

    - `.body(result.getResponse())` — set the body to the stored `PaymentResponse`.
- `return ResponseEntity.ok(result.getResponse());` — the normal path: a fresh payment returns a plain 200 OK with the response body and *no* `X-Cache-Hit` header.

4.7 application.properties — Configuration

What it is: Spring Boot's configuration file. **Why it exists:** to hold settings without hard-coding them in Java.

```
spring.application.name=idempotency
```

It currently sets only the application's name. Everything else — the port 8080, the JSON converter, Tomcat — runs on Spring Boot's defaults. (If you ever wanted a different port you would add `server.port=9090` here.)

4.8 IdempotencyApplicationTests.java — The Test

```
@SpringBootTest



class IdempotencyApplicationTests {



    @Test



    void contextLoads() {



    }
```

```
}
```

What it is: the default test class generated by Spring Initializr. `@SpringBootTest` tells the test framework to start the full Spring application context. The single `@Test` method `contextLoads()` is empty on purpose: if the application context fails to start (a broken bean, a misconfiguration), this test fails. So it is a basic **smoke test** — it proves the app can boot. It does not yet test the idempotency logic itself; adding such tests would be a natural next improvement (see Phase 6).

Phase 5 — Request Flow Simulation

We now trace three real scenarios through the actual code, line by line, including what happens in memory.

Scenario 1 — The First Payment Request

The request the client sends:

```
POST /process-payment  HTTP/1.1



Host: localhost:8080



Content-Type: application/json



Idempotency-Key: abc123









{



  "amount": 100,
```

```
  "currency": "GHS"




}
```

The code path:

1. Tomcat receives the POST and routes it to `PaymentController.processPayment`.
2. Spring fills `key = "abc123"` from the header, and builds `request` as a `PaymentRequests` with `amount = 100.0`, `currency = "GHS"`.
3. The controller calls `paymentService.processPayment("abc123", request)`.
4. In the service: `paymentStore.exists("abc123")` → the map is empty → returns `false`. The whole duplicate branch is skipped.
5. `Thread.sleep(2000)` — the thread pauses 2 seconds (simulated payment work).
6. `response = new PaymentResponse("Charged 100.0 GHS")`.
7. `payment = new StoredPayment(response, request)` — bundles them.
8. `paymentStore.save("abc123", payment)` — **the map now contains:** `{ "abc123" => StoredPayment(request=100.0/GHS, response="Charged 100.0 GHS") }`.
9. The service returns `new PaymentResult(response, false)`.
10. Back in the controller: `result.isCacheHit()` is `false`, so it returns `ResponseEntity.ok(result.getResponse())`.

The response the client receives:

```
HTTP/1.1 200 OK



Content-Type: application/json




```

```
{


  "message": "Charged 100.0 GHS"


}
```

What happened in memory: the map went from empty to holding one entry. The customer was charged once. The reply took ~2 seconds.

Scenario 2 — Duplicate Request: Same Key, Same Body

This is the retry. The client did not get the first reply in time (or just resent), so it sends the *identical* request again.

```
POST /process-payment  HTTP/1.1



Idempotency-Key: abc123          <-- SAME key as Scenario 1









{


  "amount": 100,
```

```
  "currency": "GHS"             <-- SAME body




}
```

The code path:

1. Controller fills `key = "abc123"`, `request = 100.0 / "GHS"`, calls the service.
2. In the service: `paymentStore.exists("abc123")` → the map already has this key → returns `true`. We **enter the duplicate branch.**
3. `StoredPayment saved = paymentStore.get("abc123")` — fetches the record from Scenario 1.
4. The conflict check runs:
    - `saved.getRequest().getAmount() == requests.getAmount()` → `100.0 == 100.0` → `true`.
    - `saved.getRequest().getCurrency().equals(requests.getCurrency())` → `"GHS".equals("GHS")` → `true`.
    - `true && true` → `true` — the bodies match.
5. `return new PaymentResult(saved.getResponse(), true)` — returns the **stored** response with `cacheHit = true`. Notice: **no `Thread.sleep`, no new charge, no new save.** The method exits immediately.
6. Back in the controller: `result.isCacheHit()` is `true`, so it builds `ResponseEntity.ok().header("X-Cache-Hit","true").body(...)`.

The response the client receives:

```
HTTP/1.1 200 OK




Content-Type: application/json




X-Cache-Hit: true             <-- the replay signal
```

```
{
  "message": "Charged 100.0 GHS"
}
```

What happened in memory: *nothing changed* — the map still has exactly one entry. The customer was **not** charged again. The reply was instant (no 2-second wait). The `X-Cache-Hit: true` header tells the caller "this is a replay." **This is idempotency working — the whole point of the project.**

Scenario 3 — Same Key, Different Body

Now something is wrong. A request arrives reusing key `abc123` but with a *different* amount. This could be a client bug, or an attempt to misuse the key.

```
POST /process-payment  HTTP/1.1

Idempotency-Key: abc123          <-- SAME key

{
```

```
  "amount": 999,                <-- DIFFERENT amount




  "currency": "GHS"




}
```

The code path:

1. Controller fills `key = "abc123"`, `request = 999.0 / "GHS"`, calls the service.
2. `paymentStore.exists("abc123")` → `true` → enter the duplicate branch.
3. `saved = paymentStore.get("abc123")` — the stored record still holds `amount = 100.0`.
4. The conflict check:
    - `100.0 == 999.0` → `false`.
    - The amount already failed, so `false && (anything)` → `false` — the bodies do **not** match.
5. The `else` runs: `throw new ResponseStatusException(HttpStatus.CONFLICT, "Idempotency key already used for a different request body.")`. The method stops instantly — no charge, no save.
6. Spring catches the exception and converts it into an HTTP error response. The controller's later lines never run.

The response the client receives:

```
HTTP/1.1 409 Conflict




Content-Type: application/json
```

```
{


  "status": 409,


  "error": "Conflict",


  "message": "Idempotency key already used for a different
request body."


}
```


What happened in memory: *nothing changed* — the stored record was protected, not overwritten. The 409 status tells the client "your request conflicts with an existing one." This guards the integrity of the idempotency key: one key may represent exactly one payment, forever.

1				
First request	new	—	Yes	Yes (once)

2	Duplicate	seen	same	No	No (replay)
3	Conflict	seen	different	No	No

Phase 6 — Mock Interview

Below are likely questions grouped by topic, each with a strong, beginner-friendly answer. Read the answer, then practise saying it in your own words.

6.1 Project & Idempotency Questions

Q: Explain your project in one minute.

A: "It is a Spring Boot REST API called the Idempotency Gateway. It solves the double-charging problem in payments. When the network is unreliable, a client may send the same payment twice. My gateway makes every payment carry a unique `Idempotency-Key` in a header. The first request with a key is processed and its result is saved. Any later request with the same key just gets the saved result back instead of charging again. If the same key arrives with a different payment body, I reject it with a 409 Conflict. So the customer is always charged exactly once."

Q: What does idempotency mean?

A: "It means doing an operation many times has the same effect as doing it once. Charging money is naturally not idempotent — each charge takes money. My project adds idempotency on top of it by remembering, with a key, which payments were already processed."

Q: Why does double charging happen, and why fix it on the server?

A: "It happens because of network timeouts, automatic client retries, and slow responses. The client cannot tell whether its request failed or just the reply got lost — both look like silence — so it retries. Only the server knows whether it already processed the request, so the fix has to live on the server."

Q: Why put the key in a header instead of the body?

A: "The key is metadata *about* the request, not part of the payment data itself. Headers are the natural place for metadata. It is also the convention used by real systems like Stripe, which use an `Idempotency-Key` header."

Q: Why do you also compare the request body?

A: "Because a key alone is not enough. If a key were reused for a genuinely different payment — a different amount — replaying the old answer would be wrong and would hide a real transaction. So I store the original request and compare it. Same key plus same body means a true retry; same key plus different body is a conflict, so I return 409."

6.2 Spring Boot Questions

Q: What is Spring Boot, and why use it?

A: "Spring Boot is a framework that makes building Java backends fast. It removes manual configuration with auto-configuration, includes an embedded Tomcat server so I don't install one, and lets me add features with one-line starter dependencies. I could write a working REST API quickly and focus on my actual logic."

Q: What is dependency injection? Did you use it?

A: "Dependency injection means a class declares what it needs and the framework supplies it, instead of the class building its own dependencies. I used constructor injection everywhere — my controller's constructor takes a `PaymentService`, and my service's constructor takes a `PaymentStore`. Spring creates those objects as beans at startup and passes them in. It keeps

classes loosely coupled and easy to test."

Q: What is the difference between @Component, @Service, and @RestController?

A: "They all register a class as a Spring-managed bean. `@Component` is the generic one. `@Service` and `@RestController` are specialised versions that also document intent — `@Service` means business logic, `@RestController` means a web request handler whose return values become the response body. I used `@Component` on my store, `@Service` on my service, and `@RestController` on my controller."

Q: What does @SpringBootApplication do?

A: "It is three annotations in one: it marks the configuration class, enables auto-configuration, and turns on component scanning so Spring finds my controller, service, and store automatically."

Q: What is the difference between @RequestBody and @RequestHeader?

A: "`@RequestHeader` pulls a value from an HTTP header — I use it to read the `Idempotency-Key`. `@RequestBody` converts the JSON body of the request into a Java object — I use it to turn the payment JSON into a `PaymentRequests` object."

6.3 API & HTTP Questions

Q: Why POST and not GET for the payment endpoint?

A: "Because processing a payment is an action that changes state — it moves money. GET is for reading data and should not change anything. POST is the correct verb for performing an action, and POST also carries a request body, which I need for the payment details."

Q: What status codes do you return and why?

A: "200 OK for a successful payment and also for a successful replay of a stored one. 409 Conflict when a key is reused with a different body — 409 specifically means the request conflicts with existing state, which is exactly the situation."

Q: What is REST? Is your API RESTful?

A: "REST is a style of API design: organise around resources named by URLs, use standard HTTP methods, exchange JSON, and report outcomes with HTTP status codes. My API follows that — it exposes endpoints over HTTP, uses POST, speaks JSON, and returns proper status codes — so yes, it is a small RESTful API."

Q: What is the X-Cache-Hit header?

A: "It is a custom header I add. When a response is a replay of an already-processed payment, I set `X-Cache-Hit: true`. It lets the client and anyone debugging clearly see 'this was served from memory, not freshly charged.' It was my own chosen extra feature."

6.4 Java & Architecture Questions

Q: Why ConcurrentHashMap instead of HashMap?

A: "A web server handles many requests at the same time on different threads. A plain `HashMap` is not thread-safe — concurrent access can corrupt it. `ConcurrentHashMap` is built for safe, efficient use by many threads at once. Since a payment gateway gets bursts of concurrent and duplicate requests, it is the right choice."

Q: Why did you compare currency with .equals() but amount with ==?

A: "`amount` is a primitive `double`, so `==` compares the actual numbers. `currency` is a `String`, which is an object; `==` would compare object identity, not the text. `.equals()` compares the text content, which is what I want."

Q: Describe your architecture.

A: "Four layers. The controller handles HTTP. The service holds the idempotency business logic. The store handles data — it is the in-memory memory. The model layer holds plain data classes. Each layer has one job — that is separation of concerns. The flow is client → controller → service → store and back."

Q: Why separate the store into its own layer?

A: "So the storage mechanism is swappable. Today it is a `ConcurrentHashMap` in memory. To move to a real database like Redis I would change only the store class — the service and controller would not change at all, because they only depend on the store's methods, not on how it stores data."

6.5 Honest "Weakness / Improvement" Questions

Interviewers love asking what you would improve. Answering well shows maturity. Be honest and specific:

Q: What are the limitations of this project?

A: "Three main ones, and I know how I'd address each. First, the store is in memory, so data is lost on restart and not shared across multiple instances — I would move it to Redis with a time-to-live on each key. Second, there are no automated tests for the idempotency logic yet — only the

default context-loads test — so I would add tests for all three scenarios. Third, there is no input validation; a missing currency could cause a null error — I would add validation and a global exception handler so all errors return clean JSON."

Q: Is there a concurrency edge case?

A: "Yes — if two identical requests arrive at almost the same instant, both could pass the `exists` check before either has saved, so both would process. The clean fix is an atomic operation like `putIfAbsent` on the map, so only the first wins. It is a great example of why I chose `ConcurrentHashMap` — it gives me those atomic methods to build on."

Q: Why a 2-second sleep?

A: "It is a deliberate simulation of slow real-world payment work, like contacting a bank. It also makes the demo realistic, because it creates the time window in which a client would time out and retry — which is exactly what lets me show the idempotency working. In a real system that line would be replaced by the actual payment call."

6.6 Final Interview Advice

- **Lead with the problem.** Always start "this prevents double charging" before any code detail. Interviewers care that you understand *why*.
- **Use the request flow.** If you get stuck, narrate client → controller → service → store. It always gives you something correct to say.
- **It is fine to say "I don't know, but here is how I'd find out."** That is better than guessing.
- **Own the trade-offs.** Saying "I used in-memory storage to stay focused; Redis would be the production choice" sounds far stronger than pretending the project is perfect.
- **Speak slowly and in plain words.** You understand this project — let that show by being calm, not by rushing jargon.

Phase 7 — 5-Minute Presentation Script

Below is a word-for-word script for a roughly 5-minute video or in-person presentation. It is written to sound natural and confident but still beginner-friendly. Practise it out loud two or three times. Pause at each paragraph break.

Speaking Script

[0:00 - 0:40] Opening & the Problem

"Hi, my name is Nsumba Herve, and I'd like to walk you through my project: the Idempotency Gateway, which I also call the Pay-Once Protocol.

Let me start with the real-world problem it solves. Imagine you pay 100 cedis with a mobile money app. The internet is slow that day, so your phone doesn't get a reply. It assumes the payment failed and automatically sends it again. But the first payment may have actually succeeded — so now you've been charged twice for one purchase. That's called double charging, and in payment systems it's a serious bug. My project is the safety layer that prevents it."

[0:40 - 1:30] The Core Idea: Idempotency

"The solution is a concept called idempotency. Idempotency just means that doing something many times has the same effect as doing it once — like pressing an elevator button ten times still brings one elevator.

The way I make payments idempotent is with an idempotency key. The client generates one unique key for a payment and sends it in an HTTP header called `Idempotency-Key` — on the first request and on every retry. My server uses that key as a memory. The first time it sees a key, it processes the payment and saves the result. Every time it sees that same key again, it doesn't charge again — it just returns the saved result. So the customer is always charged exactly once. This is the same approach used by real payment companies like Stripe."

[1:30 - 2:45] The Architecture

"I built this with Spring Boot, Java 17, and Maven, as a REST API. I designed it in clean, separated layers, because each layer should have only one job.

The Controller layer is the front desk — it handles the HTTP request: it reads the idempotency

key from the header and the payment data from the JSON body. The Service layer is the brain — it holds all the idempotency rules and makes the decisions. The Store layer is the memory — it saves and looks up payments using a `ConcurrentHashMap`, which I chose because a web server handles many requests at once and that map is thread-safe. And the Model layer holds the simple data classes that describe what a request, a response, and a stored payment look like.

So a request flows like this: client, to controller, to service, to store, and the response comes back the same way. Keeping these layers separate means I could, for example, swap the in-memory store for a real database like Redis and only change one file."

[2:45 - 4:00] How It Behaves — Three Cases

"My gateway handles three situations.

First, a brand-new request. The key has never been seen, so the service processes the payment — I simulate the real work with a short delay — builds the response, saves it under the key, and returns a normal 200 OK.

Second, a duplicate request — the same key and the same payment details. The service sees the key already exists, so it does not charge again. It instantly returns the saved response, and it adds a custom header, `X-Cache-Hit: true`, so anyone can clearly see this was a replay, not a new charge. That header was my own chosen extra feature.

Third, a conflict — the same key but a different payment body, like a different amount. That's dangerous, because one key should only ever mean one payment. So I store the original request and compare it. If it doesn't match, I reject the request with a 409 Conflict status, which is the correct HTTP code for 'this conflicts with something that already exists.'"

[4:00 - 4:40] What I Learned & My Contribution

"Building this taught me a lot of backend fundamentals: how REST APIs and HTTP work, how Spring Boot uses dependency injection to wire classes together, why thread safety matters on a server, and how to design code in clean, separate layers.

I also thought about the project's limitations honestly. The store is in memory, so for production I would move it to Redis with an expiry time on each key. I would add automated tests for all three scenarios, and I would handle the rare case where two identical requests arrive at the exact same instant by using an atomic map operation."

[4:40 - 5:00] Conclusion

"To sum up: my Idempotency Gateway is a Spring Boot REST API that makes payment requests safe to retry, so a customer is never accidentally charged twice. It's a small project, but it taught

me how real fintech systems protect people's money. Thank you for watching."

Delivery Tips

- **Pace:** speak a little slower than feels natural. Five minutes is plenty of time.
- **Show, don't only tell:** if it is a screen recording, run the app, send a first request in Postman, then resend the duplicate and point at the `X-Cache-Hit: true` header appearing.
- **Energy:** sound interested in your own project — the problem (people losing money) is genuinely interesting.
- **If you stumble, keep going.** A small pause is invisible to the viewer; stopping to restart is not.
- **End cleanly** with the one-line summary. Strong first and last sentences are what people remember.

Closing Note

You built a real, working solution to a real fintech problem, and you structured it with professional layering. Walk in knowing that. If you can explain the three scenarios in Phase 5 and the request flow in Phase 2, you understand this project as its owner. Good luck.